

Introduction to C++ Multithreading: Concurrency Within a Single Process

Threads share memory — that's their superpower and their curse. A complete C++ walkthrough of multithreading, race conditions, and what comes next.

10 min read · 1 day ago



Sagar

Following ▾



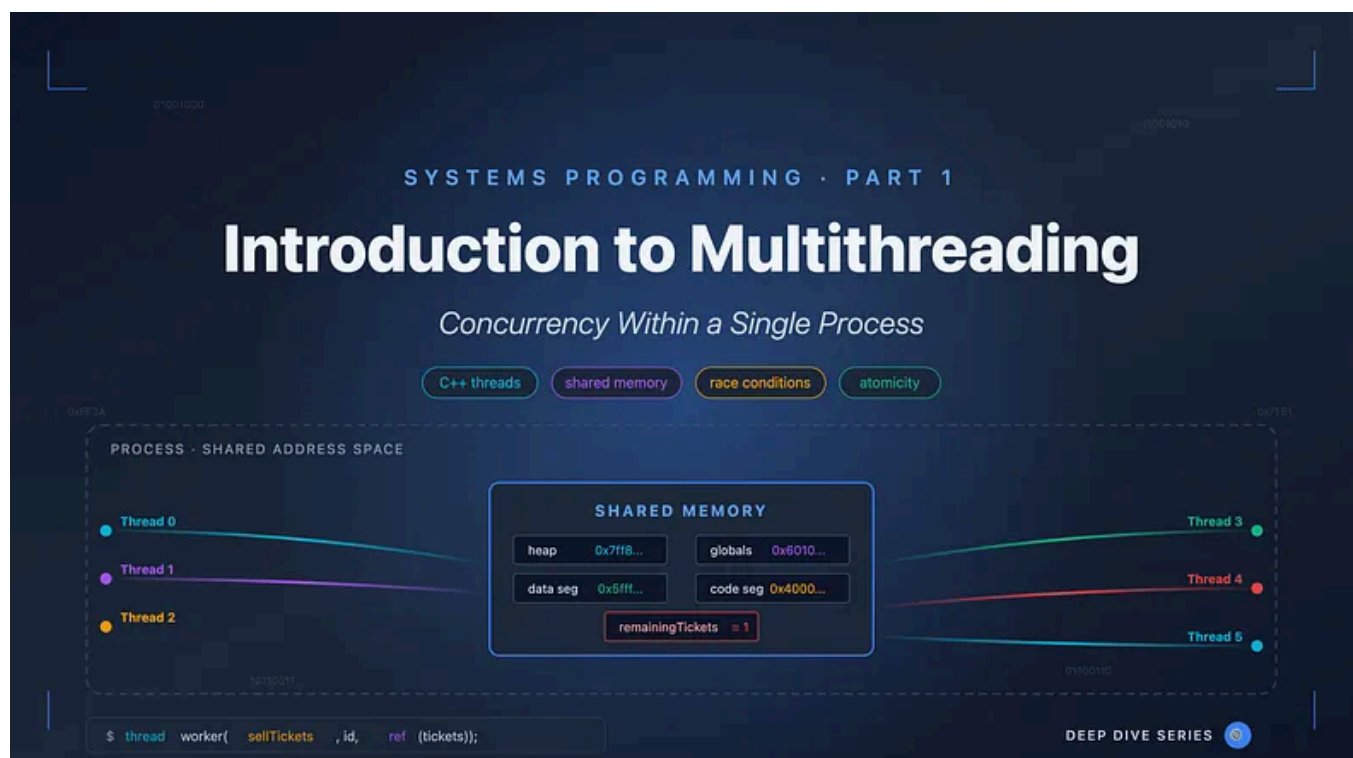
Listen



Share



More



Concurrency is one of the most transformative ideas in systems programming. You have almost certainly used it — every time a web server handles thousands of connections simultaneously, or your IDE runs a background linter while you type, concurrency is at work. In this post, we move beyond multiprocessing and into **multithreading**: *the ability to run multiple execution sequences inside a single process.*

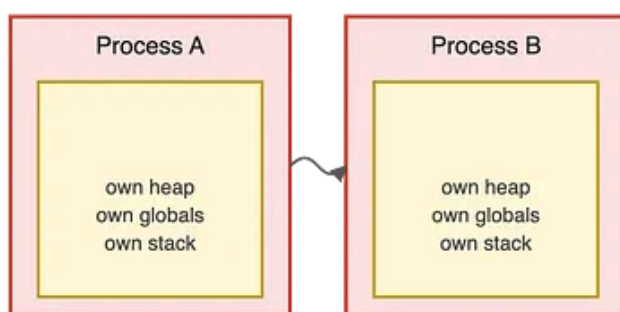
By the end of this tutorial you will understand what threads are, how they differ from processes, how to create and join them in C++, and — crucially — why shared memory makes threads simultaneously powerful and dangerous.

Why Not Just Use Processes?

You may already be familiar with `fork()` and multiprocessing. Spawning child processes is a legitimate concurrency strategy: each child has its own isolated virtual address space, you can wait on them with `waitpid()`, and you can communicate through pipes or signals.

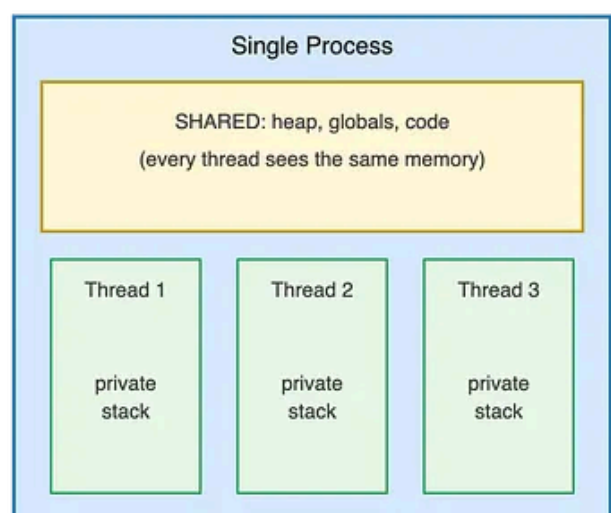
But that isolation is a double-edged sword. Here is a Processes vs. Threads — at a Glance:

Multiprocessing — isolated address spaces



pipes / signals (slow, cumbersome)

Multithreading — shared address space



Multiprocessing

Separate virtual address spaces (secure, stable)

Communication via pipes/signals (cumbersome)

Easy to run external programs (`fork` -exec)

Hard to coordinate tightly coupled tasks

Multithreading

Shared virtual address space (flexible, fast)

Communication via shared variables (easy)

Cannot easily run external programs

Easy to coordinate tightly coupled tasks

When tasks are *tightly coupled* — they need to read and write the same data, they need to be coordinated in real time — multiprocessing turns into a bureaucratic nightmare of serialization and IPC. Threads were designed precisely for this situation.

What Is a Thread?

A **thread** is an independent execution sequence within a single process. Think of a process as a restaurant kitchen and threads as the individual cooks — they all share the same kitchen (memory, globals, heap), but each has their own cutting board and recipe to follow (stack, registers, program counter).

More precisely:

- Threads run multiple functions in your program **concurrently**.
- A **thread manager** (part of the OS or runtime) switches between threads, much like the OS scheduler switches between processes. This is why threads are sometimes called *lightweight processes*.
- All threads in a process **share the same virtual address space** — including globals, the text segment, the data segment, and the heap.
- The **stack segment** is divided into a private “mini-stack” per thread. Each thread’s local variables live independently.

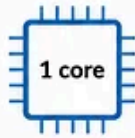
Concurrency vs. Parallelism

Two terms that are often conflated, but mean different things:

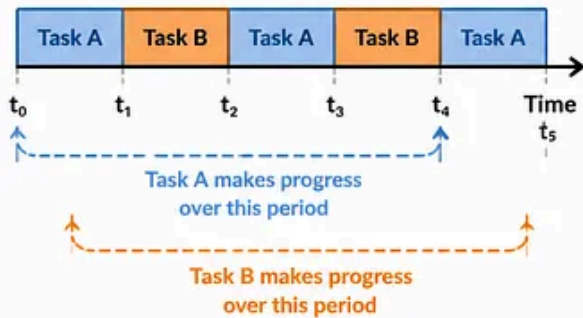
- **Concurrency** — multiple execution sequences make progress during overlapping time periods. They may take turns on a single core.
- **Parallelism** — multiple execution sequences *literally* run at the same instant on different CPU cores.

Concurrency

Tasks take turns on a single core.
They overlap in time but do not run simultaneously.



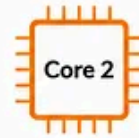
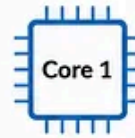
Timeline (single core)



Tasks overlap in time
but only one runs at a time
on the single core.

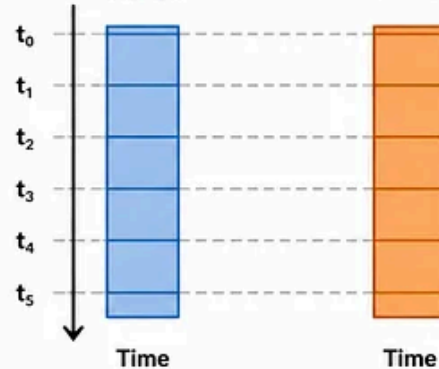
Parallelism

Tasks run at the same instant
on different cores.



Task A

Task B



Tasks run simultaneously
at the same instant
on separate cores.



Concurrency = overlapping progress



Parallelism = simultaneous execution

Threads enable concurrency even on a single-core machine, where the thread manager interleaves them rapidly enough to feel simultaneous. On a multi-core machine, threads additionally achieve true parallelism — genuine simultaneous execution across cores. Most real-world multithreaded programs rely on both.

Threads in C++: The Basics

C++ (since C++11) provides the `std::thread` class in `<thread>`. Spawning a thread is straightforward:

```
thread myThread(myFunc, arg1, arg2, ...);
```

- **myFunc** is the function the new thread will execute asynchronously.

- `arg1`, `arg2`, ... are any arguments to pass to that function.
- As soon as this line executes, the thread *may* start running — you have **no control** over exactly when.
- The return value of `myFunc` is ignored. To get results back, pass them by reference.

Waiting for a Thread to Finish

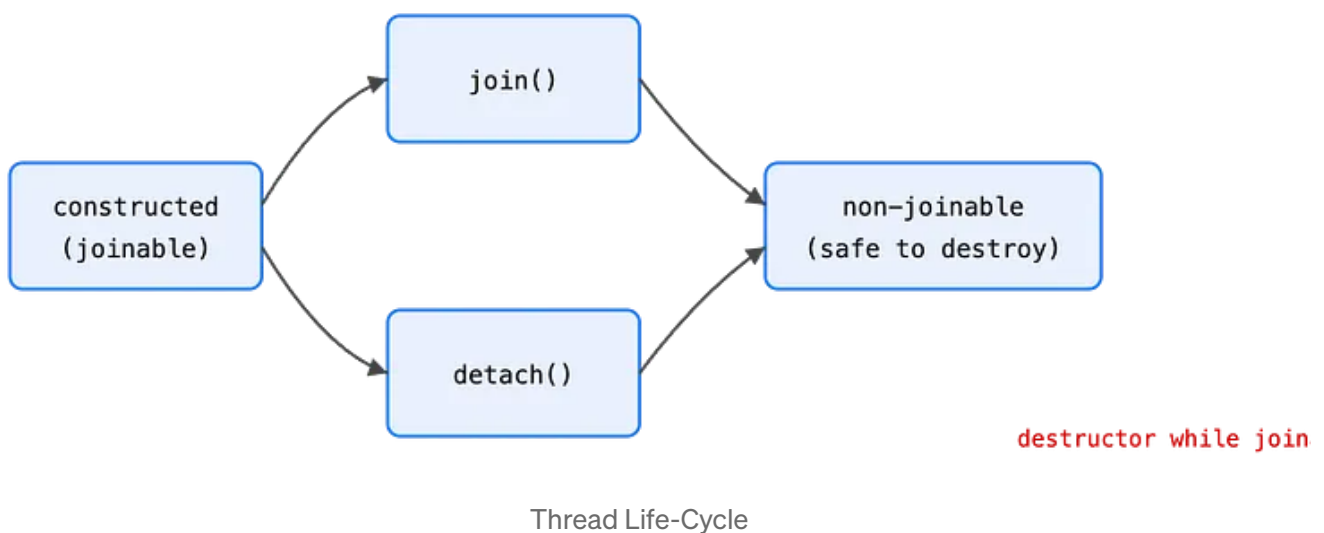
Like `waitpid()` for processes, threads have `.join()`:

```
thread myThread(myFunc, arg1, arg2);

// ... do your own work here ...

myThread.join(); // blocks until myThread finishes
```

If a `std::thread` object is destroyed while still joinable, the program calls `std::terminate()`. Always either `join()` or `detach()` every thread you create.



join VS detach :

Calling `.join()` blocks the current thread until the target finishes — the normal choice when you need the result or need to know it completed.

Calling `.detach()` releases the thread to run independently in the background, surrendering any ability to synchronize with its completion.

Beginners should default to `join()` — detached threads are harder to reason about and easier to misuse.

A modern note:

C++20 introduced `std::jthread`, which automatically calls `join()` in its destructor — so forgetting to join no longer terminates your program. If you're on C++20 or later, prefer `jthread` over `thread` for safer, RAII-friendly concurrency. I've covered this in detail in a previous post: [C++20 `std::jthread`: The Thread You Always Wanted \(No More Manual Joins\)](#).

Managing Multiple Threads

For a fleet of threads, use an array:

```
thread workers[5];

// Spawn all threads
for (size_t i = 0; i < 5; i++) {
    workers[i] = thread(myFunc, arg1, arg2);
}

// Wait for all threads to finish
for (size_t i = 0; i < 5; i++) {
    workers[i].join();
}
```

Or equivalently, with a range-based loop (note the `thread&` — thread objects cannot be copied):

```
thread workers[5];
for (thread& w : workers) {
    w = thread(myFunc, arg1, arg2);
}
```

Time to write our first thread program. Let's write.

Here is a complete, minimal threads program that spawns 6 threads, each printing a

greeting:

```
#include <iostream>
#include <thread>
using namespace std;

static const size_t kNumFriends = 6;

static void greeting(size_t i) {
    cout << "Hello, world! I am thread " << i << endl;
}

int main(int argc, char *argv[]) {
    cout << "Let's hear from " << kNumFriends << " threads." << endl;

    thread friends[kNumFriends];

    // Spawn
    for (size_t i = 0; i < kNumFriends; i++) {
        friends[i] = thread(greeting, i);
    }

    // Join
    for (size_t i = 0; i < kNumFriends; i++) {
        friends[i].join();
    }

    cout << "Everyone's said hello!" << endl;
    return 0;
}
```

Run this and you will notice something interesting: *the output order changes every run*. Thread 3 might print before Thread 0. Thread 5 might sneak in before Thread 2. This is expected and intentional — threads are scheduled independently, and the OS *makes no guarantees about ordering*.

Race Conditions and Thread Safety

The unpredictable ordering described above is not merely a curiosity. It is the source of one of the most vexing bug categories in software: **race conditions**.

A **race condition** is an unpredictable ordering of events where some orderings produce incorrect or undesired behavior.

Refer this medium article for more detailed explanation: [Atomic Operations, Race Conditions, and Critical Sections — A Clear Introduction](#)

A function is called **thread-safe** if it always executes correctly, even when called concurrently by multiple threads.



A Subtle Bug: `cout` and Interleaving:

Individual operations on `std::cout` are synchronized against data races by the C++ standard, so using `cout` from multiple threads will not produce undefined behavior. What the standard *does not guarantee* is that **compound output expressions are atomic**. Threads can still interleave their writes mid-expression, producing garbled output like:

```
Hello, world! Hello, world! I am thread I am thread 3
0
```

This fix is to wrap the entire output operation in a mutual exclusion zone — more on this concept (mutexes) in a separate post.

Threads Share Memory

Unlike parent/child processes (which get separate address spaces via copy-on-write), threads operate in the **exact same virtual address space**. This is threads' greatest strength and greatest danger.

It means you can pass a reference to a variable and have all threads read from and write to that same memory location:

```
static void greeting(size_t& i) {
    cout << "I see i = " << i << endl << osunlock;
}

for (size_t i = 0; i < kNumFriends; i++) {
    friends[i] = thread(greeting, ref(i)); // ref() required for references
}
```

This is where most beginners get burned. Arguments to `std::thread` are **copied (or moved)** into internal storage, even if the function takes references.

The following code won't compile:

```
void update(std::string& s) { s += " modified"; }

std::string data = "hello";
std::thread t(update, data); // ERROR: won't compile
                             // (refuses to bind & to a copy)
t.join();
```

You need `std::ref`:

```
std::thread t(update, std::ref(data));
t.join(); // data is now "hello modified"
```

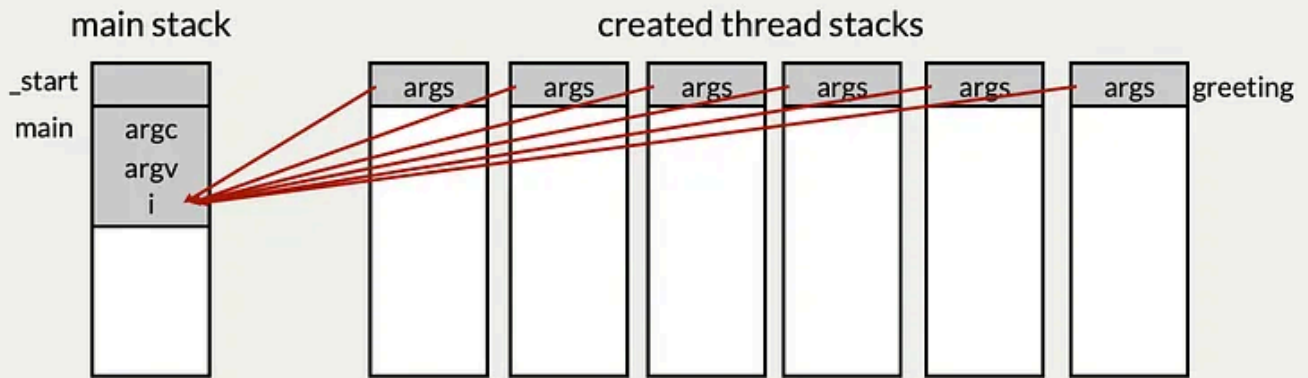
Important: When passing reference parameters to `thread()`, you must wrap the argument in `std::ref()`. The thread constructor copies its arguments by default; `ref()` forces it to pass a reference wrapper instead.

The diagram below illustrates what the memory looks like when threads share a reference to `i` from `main`'s stack:

```

1 for (size_t i = 0; i < kNumFriends; i++) {
2     friends[i] = thread(greeting, ref(i));
3 }

```



In the greeting example this is harmless — we can just pass `i` by copy instead. But the principle generalizes to all shared state, and that is where things get **dangerous**.

Thread-Level Parallelism

Threads allow a process to parallelize a problem across multiple cores.

Now let us look at a realistic use case. So we want to sell 250 concert tickets using 10 threads, each representing a ticket agent. The `main` function looks like this:

```

int main(int argc, const char *argv[]) {
    thread ticketAgents[kNumTicketAgents]; // 10 agents
    size_t remainingTickets = 250;

    for (size_t i = 0; i < kNumTicketAgents; i++) {
        ticketAgents[i] = thread(sellTickets, i, ref(remainingTickets));
    }
    for (thread& agent : ticketAgents) {
        agent.join();
    }

    cout << "Ticket selling done!" << endl;
    return 0;
}

```

And each agent thread runs:

```

static void sellTickets(size_t id, size_t& remainingTickets) {
    while (remainingTickets > 0) {
        std::this_thread::sleep_for(std::chrono::milliseconds(500)); // simulate
        remainingTickets--; // decrement the shared counter
        cout << "Thread #" << id << " sold a ticket ("
            << remainingTickets << " remain)." << endl << osunlock;
    }
    cout << "Thread #" << id
        << " sees no remaining tickets." << endl << osunlock;
}

```

Looks reasonable. But run it and you will likely see tickets oversold — **remainingTickets** might wrap around to a huge number, or agents report selling tickets that don't exist.

The Race Condition in the Ticket Code:

Here is exactly what goes wrong, step by step:

```

remainingTickets = 1

```

```

Thread #1: (line 2) while (remainingTickets > 0) → true! commits to selling
Thread #2: (line 2) while (remainingTickets > 0) → true! commits to selling
Thread #3: (line 2) while (remainingTickets > 0) → true! commits to selling

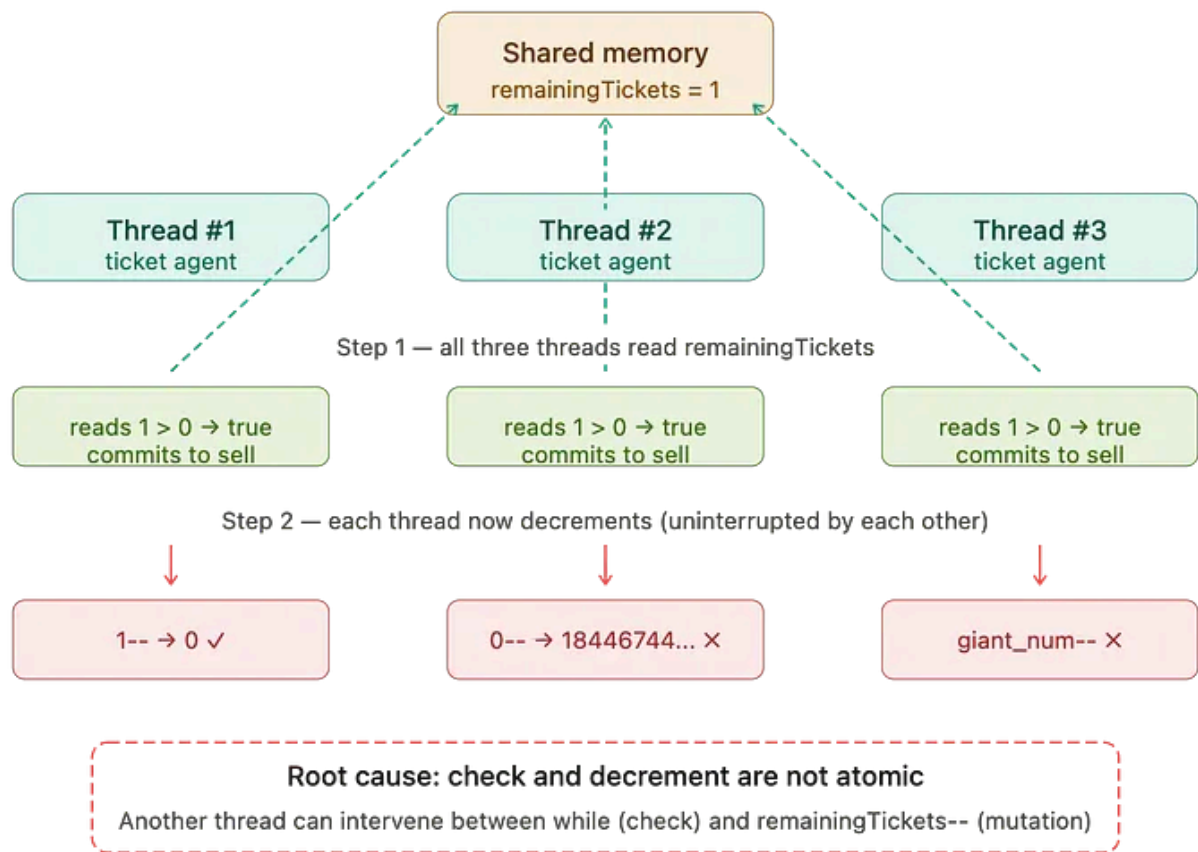
```

```

Thread #1: (line 4) remainingTickets-- → now 0
Thread #2: (line 4) remainingTickets-- → now 18446744073709551615 (!! )
Thread #3: (line 4) remainingTickets-- → now 18446744073709551614

```

Race condition: overselling tickets



Three threads all checked the condition, found `remainingTickets = 1`, and *all committed to selling*. But only one ticket existed. The check(`remainingImages > 0`) and the decrement(`remainingImages —`) are not performed atomically — another thread can intervene between the two steps.

*This is not merely a logic bug. In C++, unsynchronized concurrent reads and writes to the same memory location constitute a data race, and data races are **undefined behavior** under the C++ memory model.*

That means the compiler is permitted to make optimizations on the assumption that races never occur — which can produce behavior even stranger than a wrong counter: values that appear correct in debug builds and silently corrupt in release builds, or loops that never terminate because the optimizer cached a value in a register.

Atomicity: Why `remainingTickets--` Isn't Safe?

You might think this is fixable by combining the check and decrement into a single expression:

```
while (remainingTickets-- > 0) {  
    // ...  
}
```

But this does **not** help. C++ statements are not atomic. Even a single `--` compiles to multiple assembly instructions:

```
; Step 1: Load remainingTickets from memory into a register  
mov  -0x20(%rbp), %rax  
mov  (%rax), %eax  
  
; Step 2: Decrement the register copy  
lea  -0x1(%rax), %edx  
  
; Step 3: Store the result back to memory  
mov  -0x20(%rbp), %rax  
mov  %edx, (%rax)
```

Each core has its own registers that it has to read from.

Each thread may temporarily operate on its own locally loaded copy of the value (in registers or CPU cache) before writing changes back to memory.

If Thread A is at Step 2 and Thread B runs Steps 1–3 in full, Thread A will overwrite Thread B's update when it reaches Step 3. The decrement is **lost**.

***Atomicity** means: from the perspective of any external observer, an operation has either fully executed or not at all — no intermediate state is ever visible.*

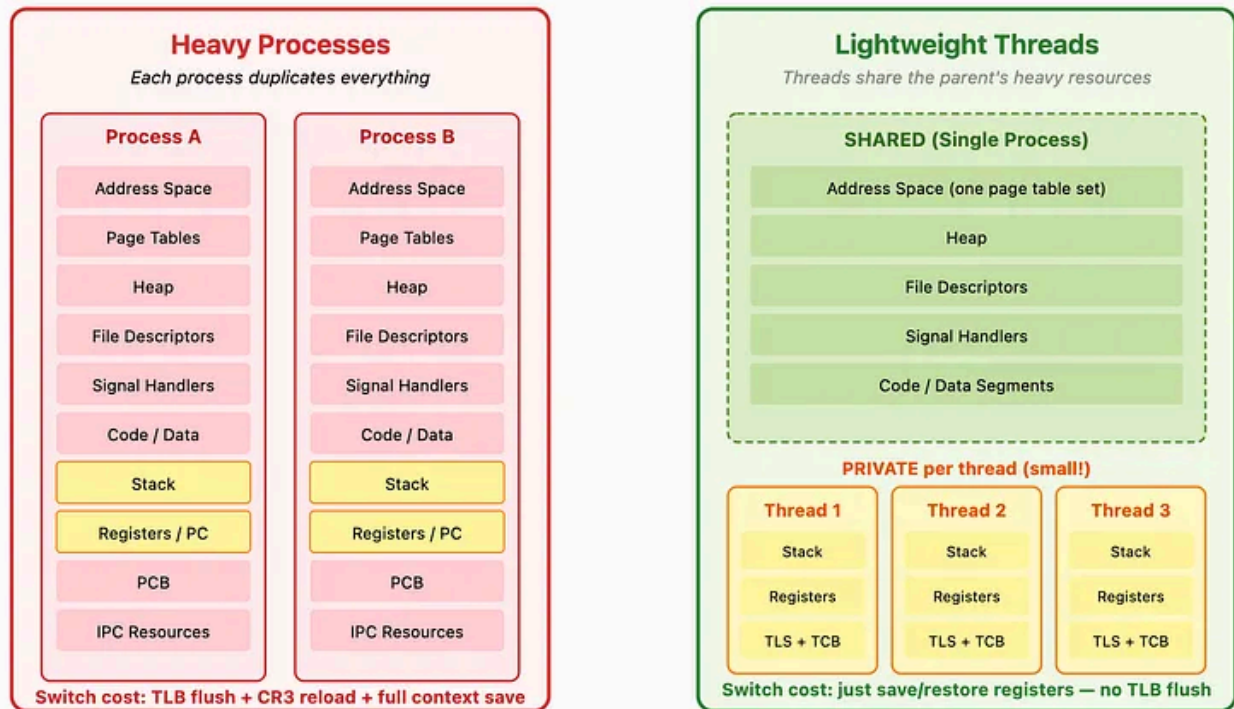
The fundamental problem is that the check (`remainingTickets > 0`) and the mutation (`remainingTickets--`) must happen as an **uninterruptible unit**. Without that guarantee, any number of threads can slip between them.

The solution — **mutexes** and **locks** — is the subject of the next post. But understanding *why* we need them requires this foundation.

Why Threads are considered light weight to Process?

Process vs Threads: Why Threads Are Lightweight

Threads share heavy resources; only stack & registers are private



Aspect	Process (Heavyweight)	Thread (Lightweight)
Definition	An independent program in execution with its own resources.	An independent execution flow within a process that shares resources.
Owns	- Virtual address space - Heap - Globals / static data - Open files, sockets, signals	- Program counter (PC) - CPU registers - Stack - Scheduling state
Shares	Nothing (each process is isolated)	Code, heap, globals, files, sockets, memory within same process
Context Switch Cost	Higher (switch address space, page tables, kernel structures)	Lower (same address space, fewer structures to switch)
Failure Impact	Isolated (usually affects only itself)	Shared (one crash may kill all threads)

Memory:	Process $\approx$ MBs	Thread $\approx$ few KBs	Creation:	<code>fork()</code> — slow	<code>clone(CLONE_VM ...)</code> — fast
Communication:	IPC (pipes, shm, sockets)	Direct shared memory	Isolation:	Strong	None (shared faults crash all)

Summary: What We Know So Far

Concept	Key Point	Thread API	Notes
Thread	Independent execution within a process; shares address space	<code>thread(func, args...)</code>	Multiple threads run concurrently in same process memory
<code>join()</code>	Blocks until that thread finishes; required on all joinable threads	<code>t.join()</code>	Ensures all threads complete before main exits
<code>detach()</code>	Releases thread to run independently; use with caution	<code>t.detach()</code>	Thread runs daemon-style; can't join later
<code>ref(x)</code>	Passes x by reference to a thread	<code>thread(func, ref(x))</code>	All threads see/modify same variable → race conditions
Race condition	Unpredictable interleaving causing incorrect behavior	-	Timing-dependent bugs; hard to reproduce
Thread-safe	Correct regardless of how many threads call it concurrently	-	Must handle all possible interleavings
Data race	Unsynchronized concurrent access to same memory — undefined behavior	-	Actual memory corruption; compiler makes no guarantees
Atomicity	An operation fully executes or not at all; no intermediate state visible	-	<code>std::atomic</code> provides this guarantee

What Comes Next?

We have seen that shared memory makes it easy to coordinate, but also easy to corrupt. The ticket-selling example shows that we need a way to enforce mutual exclusion — to say “only one thread may execute this critical section at a time.”

The tools for this are:

- **Mutexes** (`mutex`, `lock_guard`) — prevent simultaneous access to shared data
- **Condition variables** — let threads wait for a condition to become true
- **Semaphores** — generalize mutual exclusion to a count
- **Multithreading patterns** — producer/consumer, thread pools, and beyond

Closing Thoughts

Threads are powerful precisely because they replace expensive inter-process communication with direct shared-memory coordination. The same data structure, the same counter, the same queue — every thread can see it without serializing it across a pipe.

But that convenience has a price. Once memory is shared, correctness depends not just on *what* code executes, but on the *relative timing* of execution across threads. A program that works perfectly in isolation can fail intermittently and unpredictably the moment a second thread enters the picture.

Modern synchronization primitives — **mutexes**, **condition variables**, **semaphores** — exist to restore predictability to that timing. They let you carve out regions of your code where only one thread may operate at a time, turning the chaos of arbitrary interleaving back into something you can reason about.

If you're serious about passing technical interviews, try [PracHub](#). It helped me structure my preparation with advanced mock sessions. [\[Check it out here\]](#) and start practicing. (Disclosure: This is an affiliate link).

Found this article helpful ? Clap 🙌 and follow me for more C++ and system programming content.

Programming

Software Development

C Plus Plus Language

System Programming

Coding



Following ▾

Written by Sagar

145 followers · 5 following